

AI ENGINEERING CANVAS

Complete Product, Architecture, Design & Technology Specification

Concept: an AI-native technical canvas for creating, understanding, modifying, animating and presenting software/system flows.

Core loop: Prompt → Diagram • Code → Diagram • Manual Edit ↔ AI Edit • Diagram → Voice Q&A; • Flow → Animation

Build target: portfolio-grade MVP first, production-grade capabilities later

Specification: 2 October 2026

This document is a roadmap and architecture contract for Codex. It deliberately separates MVP work from later features so development does not become uncontrolled.

1. Product Definition

Build an interactive engineering canvas where a developer can draw a technical flow manually, generate one from a natural-language prompt, derive one from source code, modify it through chat, ask questions about the system, and eventually explore it through live voice. The diagram is stored as structured graph data rather than an image.

Primary users: software developers, tech leads, architects, students, interview candidates and anyone who needs to explain technical systems.

Primary problem: creating and explaining architecture/flows is slow; static diagrams do not understand the underlying code and are difficult to explore interactively.

2. MVP Scope

- Manual node/edge canvas.
- Project and diagram persistence.
- Prompt → structured diagram generation.
- Natural-language editing of an existing diagram.
- Diagram version history.
- Node/edge inspector.
- Basic TypeScript/JavaScript code import and analysis.
- Source references on generated nodes.
- AI explanation of selected nodes.
- Real-time UI events through Socket.IO.

3. Later Scope

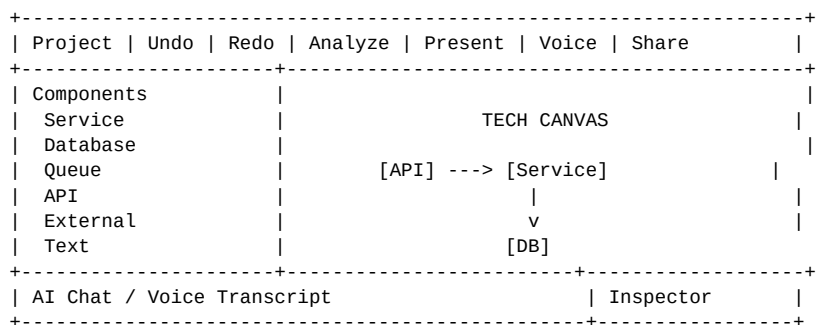
- Live Vapi voice sessions.
- Voice-triggered highlighting.
- Failure-path analysis.
- Java/Spring enrichment.
- GitHub repository integration.
- Animated presentation mode.
- RabbitMQ worker architecture for large analyses.
- OpenTelemetry/Langfuse observability.
- Collaboration, sharing, more languages and integrations.

4. Explicit Non-Goals

- Do not clone all Excalidraw features.
- Do not support every language initially.
- Do not execute uploaded source code.
- Do not start with many microservices.
- Do not implement phone-call automation before web voice works.
- Do not let the model directly mutate frontend state.

5. User Experience

Main workspace: a large canvas, left component palette, bottom AI chat/voice area, and right inspector. The user can select a node, ask questions about it, or ask the AI to modify the current graph.



Critical UX rule: AI edits should be previewable. Example: “Added Redis between Backend and PostgreSQL; reads use Redis; writes remain direct.” The user can approve/reject the change.

6. Core Journeys

Prompt → Diagram

User prompt
-> AI planner
-> structured graph JSON
-> schema validation
-> automatic layout
-> canvas
-> manual edits

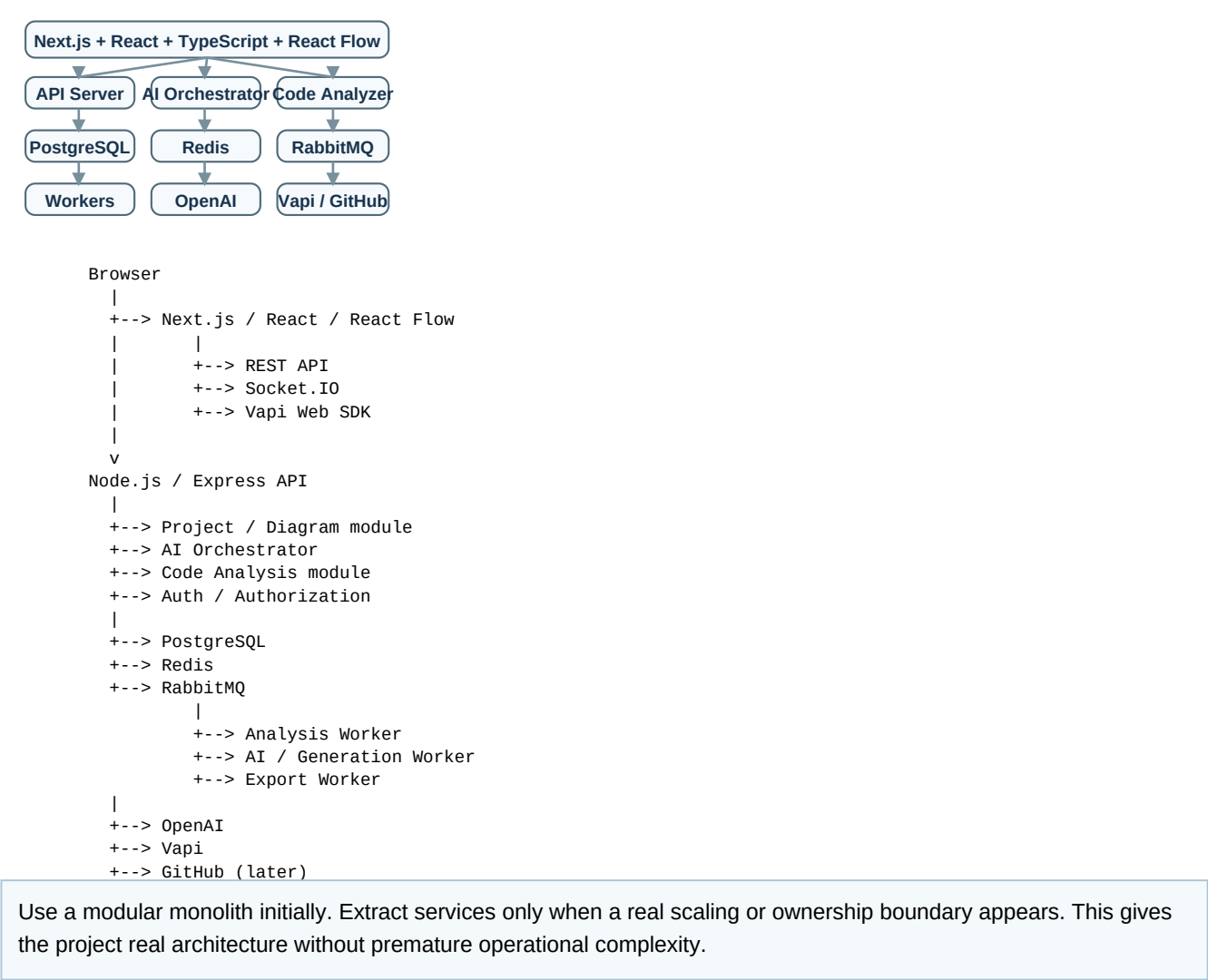
Code → Diagram

Repository / files
-> language detection
-> parser / syntax tree
-> symbols + relations
-> semantic code graph
-> AI interpretation
-> diagram graph
-> canvas

Voice → Understanding

Voice question
-> Vapi
-> agent tool call
-> code/graph retrieval
-> grounded answer
-> Socket.IO highlight event
-> spoken answer + visual highlight

7. System Architecture



8. Technology Stack

Layer	Choice	Role
Frontend	Next.js + React + TypeScript	UI, routing, canvas host
Canvas	@xyflow/react	Structured node/edge editor
UI	Tailwind CSS	Styling
Client state	Zustand	Canvas/UI state
Backend	Node.js + Express + TypeScript	REST APIs and orchestration

Realtime	Socket.IO	Progress and visual events
AI	OpenAI Agents SDK	Agents, tools, sessions, structured interactions
Voice	Vapi Web SDK	Live browser voice
Parsing	Tree-sitter	Multi-language syntax parsing
Enrichment	TypeScript compiler API / ts-morph	TS semantic analysis
DB	PostgreSQL	Users/projects/diagrams/versions/metadata
Cache	Redis	Cache/rate limits/locks/temp state
Queue	RabbitMQ	Durable background work
Observability	OpenTelemetry + Langfuse	Backend + AI tracing
Infra	Docker + Docker Compose	Local multi-container environment
Validation	Zod	Runtime API/AI schema validation
Tests	Vitest + Supertest + Playwright	Unit/API/E2E

React Flow is currently a customizable React library for node-based editors and already provides dragging, zooming, panning and selection. tldraw was considered as an alternative, but React Flow better matches the structured technical-graph requirement. [■cite■turn0search11■turn0search9■](#)

9. Diagram Data Model

The diagram is a graph document. Never store only a screenshot. AI, animation, highlighting and manual editing all operate on the same graph.

```
Diagram
  id, projectId, name, currentVersionId

DiagramVersion
  id, diagramId, versionNumber, graphJson, createdBy, createdAt

Graph
  nodes[]
  edges[]
  groups[]
  viewport{}

Node
  id, type, position, size, data, sourceRefs[]

Edge
  id, source, target, type, data, animation

SourceRef
  repositoryId, filePath, startLine, endLine, symbolKey
```

Example graph JSON

```
{
  "nodes": [{
    "id": "order-service",
    "type": "service",
    "position": {"x": 420, "y": 240},
    "data": {
      "label": "Order Service",
      "sourceRefs": ["src/order/OrderService.java"]
    }
  }],
  "edges": [{
    "id": "e1",
    "source": "api",
    "target": "order-service",
    "type": "request",
    "data": {"label": "POST /orders", "animated": true}
  }]
}
```

PostgreSQL JSONB is appropriate for flexible graph metadata while core entities and searchable relationships remain relational. PostgreSQL documents JSONB as a decomposed binary representation with indexing support. [■cite■turn1search9■](#)

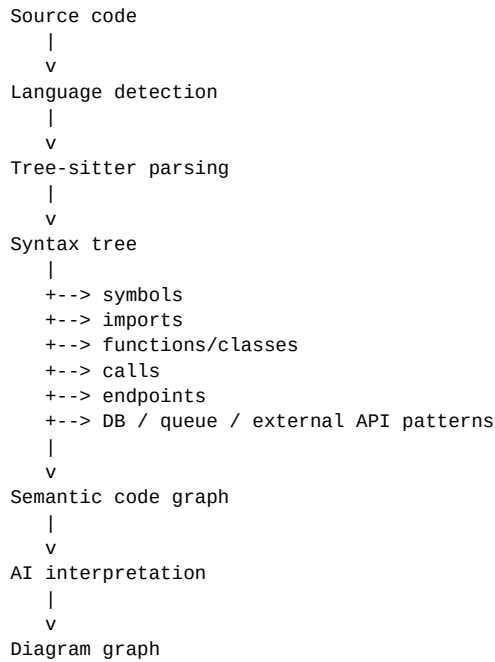
10. AI Architecture

```
User
|
v
AI Orchestrator
|
+--> Context Builder
|   +--> current graph
|   +--> selected node
|   +--> relevant code symbols
|   +--> conversation
|
+--> Tools
    get_node_details()
    get_source_reference()
    search_code()
    get_callers()
    get_callees()
    get_failure_paths()
    highlight_path()
    propose_diagram_operations()
    apply_diagram_operations()
    explain_diagram()
|
v
```

```
Typed operation list
|
v
Zod validation
|
+--> Preview
+--> Apply
|
v
New DiagramVersion
```

The current OpenAI Agents SDK for TypeScript supports agents, function tools, sessions, MCP tool integration, human-in-the-loop mechanisms and tracing. Keep our graph schema and domain tools provider-independent. [■cite■turn0search10■turn0search12■](#)

11. Code Analysis Pipeline



Start with TypeScript/JavaScript. Add Java/Spring next. Tree-sitter is an incremental parsing library intended for syntax trees across languages and remains useful with syntax errors; its query system supports targeted extraction. [■cite■turn1search2■turn1search11■](#)

For TypeScript-specific semantic information, use the compiler API or ts-morph where syntax-only analysis is insufficient. The AI should not be responsible for basic parsing that deterministic tooling can provide.

Source-grounded node example

```
Node: PaymentService.processPayment
Source: src/payment/PaymentService.java:42-78

Relations:
  OrderService -> PaymentService.processPayment
  PaymentService -> PaymentRepository.save
  PaymentService -> PaymentGateway.charge

Evidence:
  direct_call = source-detected
  runtime_dependency = inferred
```

For GitHub later, use the official REST API/contents/tree mechanisms rather than scraping repositories. [■cite■turn1search0■turn1search14■](#)

12. Live Voice

```
Browser microphone
-> Vapi Web SDK
-> Voice Agent
-> backend tools
    get_node_details
    search_code
    get_failure_paths
    highlight_path
-> spoken response
-> Socket.IO visual event
-> canvas highlight
```

Example: “What happens if PaymentService fails?” The voice agent calls the failure-path tool, the backend returns evidence from the graph/code index, the canvas highlights the affected path, and the agent explains what is known versus inferred.

Vapi currently supports browser voice interfaces and tool calls; server-backed tools are appropriate when the model needs the result for further reasoning. [■cite■turn0search1■turn0search3■](#)

13. Animation / Presentation Engine

Animation should be graph-driven. A presentation is a sequence of graph states, not a video generated ahead of time.

```
Presentation
  diagramVersionId
  steps[]

Step
  title
  description
  activeNodeIds[]
  activeEdgeIds[]
  camera {x, y, zoom}
  durationMs
  narrationText?
```

- idle — normal diagram.
- active — current node/edge emphasized.
- traversing — directional flow animation.
- completed — prior path remains visible.
- error — failure path emphasized.
- paused — preserve exact state.

Controls: play, pause, next, previous, restart, speed, camera focus. Voice narration can be added later; presentation must work independently.

14. API Design

```
POST /api/projects
GET  /api/projects
GET  /api/projects/:id
PATCH /api/projects/:id

POST /api/diagrams
GET  /api/diagrams/:id
POST /api/diagrams/:id/operations
GET  /api/diagrams/:id/history

POST /api/ai/generate-diagram
POST /api/ai/edit-diagram
POST /api/ai/explain
POST /api/ai/query

POST /api/code-sources
POST /api/code-sources/:id/analyze
GET  /api/code-sources/:id/status
GET  /api/code-sources/:id/symbols

POST /api/voice/session
POST /api/vapi/webhook

GET  /api/jobs/:id
```

Socket events: diagram:updated, ai:started, ai:tool-started, ai:completed, analysis:progress, analysis:completed, voice:highlight, presentation:step.

15. RabbitMQ / Redis / Workers

Large repository analysis, large AI generation and exports should not block an HTTP request. Queue these jobs and stream progress to the browser.

```
RabbitMQ
analysis.exchange -> code.analysis.queue -> DLQ
ai.exchange       -> diagram.generation.queue -> DLQ
export.exchange   -> export.queue -> DLQ
```

```
Worker rules:
- acknowledge after success
- bounded retries
- dead-letter permanent failures
- idempotency keys
- persistent job status
- correlation IDs
```

RabbitMQ's current documentation provides the work-queue and operational primitives needed for this architecture. [■cite■turn0search13■](#)

Redis responsibilities

- Rate limiting AI/voice usage.
- Short-lived progress/cache state.
- Distributed locks where concurrent mutations require them.
- Diagram context cache.
- Idempotency/deduplication keys.

Redis supports data structures for caching/state/event processing and documents distributed-lock patterns. Use locking only where there is a concrete concurrency requirement. [■cite■turn0search6■turn0search5■](#)

16. Observability

- Structured logs with requestId, projectId, jobId.
- OpenTelemetry traces for API, workers and external calls.
- AI traces for model/tool calls and latency.
- Metrics for queue depth, analysis duration, errors, WebSocket connections and AI usage.
- Langfuse later for AI traces, sessions and evaluation.

OpenTelemetry is a vendor-neutral framework for traces, metrics and logs. Langfuse provides AI/agent tracing and supports OpenTelemetry-based instrumentation. [■cite■turn0search0■turn1search1■turn1search6■](#)

17. Security

- Never expose private OpenAI/Vapi credentials in the browser.
- Treat uploaded repositories as untrusted.
- Never execute uploaded source code on the API server.
- Perform parsing in isolated workers/containers where practical.
- Limit repository size, file count and analysis duration.
- Redact secrets from logs and telemetry.
- Scope every project query to the authenticated owner.
- Validate every AI graph operation with Zod and graph-consistency checks.
- Store source references so important AI explanations are auditable.

18. AI Reliability

- Ground explanations in indexed symbols/source references.
- Explicitly mark uncertain or inferred behavior.
- Retrieve only relevant code/graph context instead of sending the whole repository.
- Never allow arbitrary frontend state mutation from the model.
- Store AI operations and resulting versions.
- For failure questions, distinguish statically detected handling from inferred runtime behavior.

19. Repository Structure

```
ai-engineering-canvas/  
  ■■■ apps/  
    ■ ■■■ web/  
    ■ ■■■ api/  
  ■■■ packages/  
    ■ ■■■ graph-schema/  
    ■ ■■■ ai-tools/  
    ■ ■■■ code-analysis/  
    ■ ■■■ diagram-engine/  
    ■ ■■■ shared/  
  ■■■ workers/  
    ■ ■■■ analysis-worker/  
    ■ ■■■ ai-worker/  
    ■ ■■■ export-worker/  
  ■■■ infra/  
    ■ ■■■ docker/  
    ■ ■■■ observability/  
  ■■■ docs/  
  ■■■ tests/  
  ■■■ docker-compose.yml  
  ■■■ README.md
```

Do not create every package on day one. Start with the smallest modules that make the boundary clear; extract only when needed.

20. Development Roadmap

Phase 0 — Foundation

- Next.js + React + TypeScript.
- Express + TypeScript API.
- Docker Compose: PostgreSQL, Redis, RabbitMQ.
- Basic auth/project model.
- Shared Zod graph schema.
- CI: lint, typecheck, tests.

Phase 1 — Manual Canvas

- React Flow canvas.
- Node palette and connectors.
- Groups, labels, selection.
- Save/load.
- Undo/redo.
- Versioning.

Phase 2 — Prompt → Diagram

- OpenAI generation endpoint.
- Structured graph output.
- Zod validation.
- Initial layout.
- AI chat panel.
- Preview/apply.

Phase 3 — AI Editing

- Current-graph context.
- Natural-language modifications.
- Operation diff.
- Version history.
- Explain selected node.

Phase 4 — Code → Diagram

- File/ZIP upload.
- TypeScript/JavaScript parsing.
- Symbol/relation graph.
- Node/source mapping.
- Basic architecture generation.

Phase 5 — Interactive Understanding

- Code search.

- Caller/callee tools.
- Failure-path tools.
- Highlight tools.
- Socket-driven visual responses.

Phase 6 — Voice

- Vapi Web SDK.
- Voice lifecycle.
- Server-backed reasoning tools.
- Transcript.
- Voice → visual highlight.

Phase 7 — Presentation

- Scenario paths.
- Step engine.
- Animated arrows.
- Camera focus.
- Play/pause/next/previous.

Phase 8 — Hardening

- RabbitMQ retries/DLQ.
- Redis limits/locks.
- OpenTelemetry.
- Langfuse.
- Load/failure tests.
- Security review.
- Deployment.

21. Codex Handoff — Start Here

Use this as the initial Codex instruction. It intentionally stops after the foundation and manual canvas instead of asking Codex to build the entire roadmap at once.

Build an application called AI Engineering Canvas.

PRODUCT

An AI-native technical diagramming and code-understanding application.

Users will eventually:

- draw technical flows manually
- generate diagrams from prompts
- modify diagrams using chat
- upload code and generate architecture/flows
- ask questions about the system
- use live Vapi voice to ask questions
- highlight relevant nodes/paths
- present flows with animation

FIRST STACK

Frontend:

- Next.js
- React
- TypeScript
- @xyflow/react
- Tailwind CSS
- Zustand
- Socket.IO client

Backend:

- Node.js
- Express
- TypeScript
- Zod
- Socket.IO

Infrastructure:

- PostgreSQL
- Redis
- RabbitMQ
- Docker Compose

AI:

- OpenAI Agents SDK (add after the canvas foundation is stable)

DO NOT IMPLEMENT YET

- Vapi
- code analysis
- GitHub
- collaboration
- presentation mode
- microservices

FIRST MILESTONE

1. Create repository structure.
2. Create web app.
3. Create API app.
4. Add Docker Compose for PostgreSQL, Redis and RabbitMQ.
5. Create project/diagram/version schema.
6. Implement React Flow canvas.
7. Implement node creation, movement, connection, edit and delete.
8. Implement shared graph schema using Zod.
9. Implement save/load.
10. Implement diagram versioning.
11. Add unit/API/E2E tests.
12. Document how to run locally.

RULES

- The diagram is structured graph JSON, never just an image.
- AI must eventually return typed graph operations, not frontend code.
- Every mutation must be validated.
- Every diagram mutation creates a version.
- Do not introduce microservices prematurely.
- Never execute uploaded source code.
- Keep graph schema independent of OpenAI/Vapi.
- Keep provider-specific logic behind interfaces.

When the first milestone is complete, stop and report:

- repo structure
- run commands
- environment variables
- schema
- endpoints
- tests
- remaining work

22. MVP Definition of Done

- User can create a project and open a canvas.
- User can add/move/connect/edit/delete nodes.
- User can save/reload diagrams.
- Diagram versions are persisted.
- Graph schema is shared and validated.
- Local infrastructure starts with Docker Compose.
- Core graph/API behavior has automated tests.
- Architecture is ready for AI generation without rewriting the canvas.

23. Portfolio Version Definition of Done

- Prompt → diagram.
- AI diagram editing.
- Code → code graph → diagram.
- Source references.
- Grounded AI explanations.
- Failure-path analysis.
- Live Vapi voice Q&A.;
- Voice-triggered visual highlighting.
- Animated presentation.
- RabbitMQ background jobs.
- Redis rate limiting/locking.
- OpenTelemetry/Langfuse tracing.
- Dockerized deployment.
- Security controls and repository isolation.

24. Technology Research Notes

The stack choices above were checked against current official documentation on 2 October 2026. Exact package versions should be pinned when implementation starts.

- React Flow: current documentation describes it as a customizable React node-based editor with built-in dragging, zooming, panning and selection. ■cite■turn0search11■
- tldraw: current SDK supports a full single-user canvas and was considered as an alternative freeform layer. ■cite■turn0search9■
- OpenAI Agents SDK: current TypeScript docs cover agents, tools, sessions, MCP, human-in-loop and tracing. ■cite■turn0search10■turn0search12■
- Vapi: current docs cover browser voice, web SDK, tool calls and server integrations. ■cite■turn0search1■turn0search3■
- Tree-sitter: current docs describe incremental parsing and syntax trees with broad language support. ■cite■turn1search2■turn1search16■
- RabbitMQ: current docs are for the 4.3 line and include developer/operations guidance. ■cite■turn0search13■
- Redis: current docs cover data structures, caching/state and distributed locks. ■cite■turn0search6■turn0search5■
- OpenTelemetry: current docs define vendor-neutral traces, metrics and logs. ■cite■turn0search0■
- Langfuse: current docs provide AI/agent tracing and OpenTelemetry integration. ■cite■turn1search1■turn1search6■
- Docker Compose: current docs describe multi-container application definition and lifecycle management. ■cite■turn1search4■
- GitHub REST API: current docs provide repository and contents APIs for later repository import. ■cite■turn1search0■turn1search14■
- PostgreSQL: current docs support JSONB for flexible graph metadata with indexing. ■cite■turn1search9■

25. Final Engineering Principle

Build vertical slices, not the whole vision at once. The product should become useful early: first a good canvas, then AI generation, then code understanding, then voice and animation. Every later feature should reuse the same structured graph and tool layer.

The central architectural asset is the **Engineering Graph**: nodes, edges, source references, relationships, operations and versions. The canvas, AI, code analyzer, voice agent and presentation engine should all operate against that common model.

End state: a developer can upload or describe a system, see a structured interactive architecture, ask “what happens if this fails?”, hear the answer in a live voice session, watch the relevant path highlight, modify the diagram, and present the flow step-by-step.